In looking at the irb output, you spot the problem. Class Array already has a method named replace. When you defined your own replace method, you inadvertently overwrote the original replace, a method that some other part of Bookworm was relying on.

This is the dark side to Open Classes: if you casually add bits and pieces of functionality to classes, you can end up with bugs like the one you just encountered. Some people would frown upon this kind of reckless patching of classes, and they would refer to the previous code with a derogatory name: they'd call it a *Monkeypatch*.

Spell: Monkeypatch

Now that you know what the problem is, you and Bill rename your own version of Array#replace to Array#substitute and fix both the tests and the calling code. You learned a lesson the hard way, but that didn't spoil your attitude. If anything, this incident piqued your curiosity about Ruby classes. It's time for you to learn the truth about them.

Inside the Object Model

Where you learn surprising facts about objects, classes, and constants.

Your recent experience with *Open Classes (14)* hints that there is more to Ruby classes than meets the eye. *Much* more, actually. Some of the truths about Ruby classes and the object model in general might even come as a bit of a shock when you first uncover them.

There is a lot to learn about the object model, but don't let all this theory put you off. If you understand the truth about classes and objects, you'll be well on your way to being a master of metaprogramming. Let's start with the basics: objects.

What's in an Object

Imagine running this code:

```
class MyClass
  def my_method
    @v = 1
  end
end

obj = MyClass.new
obj.class           # => MyClass
```

Look at the obj object. If you could open the Ruby interpreter and look into obj, what would you see?